Idea central

Todo gira alrededor de un **análisis** (o “job”) con ID estable:

1. Subís el PDF → se crea el análisis
2. El back procesa (extraer texto, detectar barreras, etc.)
3. El front consulta estado / recibe eventos
4. Operás barreras y propuestas
5. Generás el documento adaptado

La ** de verdad vive en el **back** (DB), no en `sessionStorage`. El front solo muestra y envía acciones.

Modelo mínimo (persistido)

```
Analysis
  id, status, originalFileUrl, adaptedFileUrl?, createdAt…

Barrier
  id, analysisId, dimension, severity, title, why, where,
  recommendation, status (pending|accepted|discarded), order

Proposal
  id, barrierId, kind (initial|variant|manual),
  text, baselineAiText?, remainingAttempts, version, isCurrent

Note (opcional)
  barrierId, text
```

Con eso podés navegar Adaptar ↔ Detalle y volver a cualquier barrera sin perder nada.

¿Cuántas APIs / endpoints?

Con **una API REST** (o REST + un canal de progreso) alcanza. No hace falta un microservicio por pantalla al principio.

Endpoints sugeridos (~10–12)

#	**Método**	**Endpoint**	**Para qué**
1	`POST`	`/analyses`	Subir PDF, crear análisis → `{ analysisId }`
2	`GET`	`/analyses/:id`	Resumen + `status` + stats
3	`GET`	`/analyses/:id/status`	Polling liviano: `uploaded | extracting |`

#	Método	Endpoint	Para qué
			`analyzing | ready | failed` + progreso por dimensión
4	`GET`	`/analyses/:id/barriers`	Lista agrupada por dimensión (lo de Revisar/Adaptar)
5	`GET`	`/analyses/:id/barriers/:barrierId`	Detalle de una barrera (Detalle)
6	`POST`	`/analyses/:id/barriers/:barrierId/proposals`	Generar propuesta IA (consume 1 intento)
7	`POST`	`/analyses/:id/barriers/:barrierId/proposals/variants`	Más simple / breve / pasos / etc.
8	`PUT`	`/analyses/:id/barriers/:barrierId/draft`	Guardar edición manual del texto
9	`POST`	`/analyses/:id/barriers/:barrierId/accept`	Aceptar propuesta actual
10	`POST`	`/analyses/:id/barriers/:barrierId/discard`	Descartar
11	`POST`	`/analyses/:id/barriers/:barrierId/reopen`	“Volver a editar”
12	`POST`	`/analyses/:id/export`	Generar PDF/DOCX adaptado
13	`GET`	`/analyses/:id/exports`	Links de descarga (Resultado)

Opcional: WebSocket/SSE `GET /analyses/:id/events` en lugar de (o además de) polling en el paso Analizar.

Eso mapea 1:1 con tu UI: Cargar → 1, Analizar → 3, Revisar/Adaptar → 4, Detalle → 5–11, Resultado → 12–13.

Flujo de proceso (paso a paso)

1. Cargar

- Front: `POST /analyses` (multipart PDF)
- Back: guarda archivo, crea registro `status=uploaded`, encola job
- Respuesta inmediata: `{ analysisId }`
- Front guarda el ID (URL: `/analisis/analizar?id=…` o store) y pasa a Analizar

2. Analizar (loading)

Jobs en background, por ejemplo:

1. Extraer texto/estructura del PDF
2. Correr criterios por dimensión
3. Persistir barreras

Front hace polling cada 1–2 s a `/status` hasta `ready` o `failed`.
El checklist por dimensión puede venir en el status:

```
{
  "status": "analyzing",
  "progress": 0.6,
  "dimensions": {
    "comprension": "done",
    "estructura": "running",
    "visual": "pending"
  }
}
```

3. Revisar

- `GET /barriers` → mismos datos que tu `BARRIERS`
- Stats se calculan en el back (totales, severidad alta, dimensiones afectadas)

4. Adaptar

- Misma lista; cada barrera tiene `status` e `attemptsRemaining`
- “Generar propuesta” → endpoint 6 → cuando termina, front navega a Detalle con `barrierId`
- “Adaptar a mano” → Detalle sin propuesta (o draft vacío)
- “Volver a editar” → `reopen` + Detalle

5. Detalle

Todo lo que hoy está en sesión local pasa a ser:

- draft en `PUT /draft`

- propuestas/variantes en 6–7
- accept / discard / reopen en 9–11

El front solo necesita `analysisId` + `barrierId` en la URL; al recargar, el back devuelve el estado.

6. Resultado

- `POST /export` aplica solo barreras `accepted`, genera el documento nuevo
- Status del export: `queued` → `generating` → `ready`
- `GET /exports` para los archivos

Cómo queda la navegación (sin sessionStorage)

URL sugerida:

```
/analisis/analizar?analysisId=…
/analisis/revisar?analysisId=…
/analisis/adaptar?analysisId=…
/analisis/detalle?analysisId=…&barrierId=…
/analisis/resultado?analysisId=…
```

Cada pantalla hace `GET` al montar.
Ctrl+Shift+R sigue mostrando lo mismo porque está en DB, no en el navegador.

Arquitectura interna del back (simple)

```
API (Next/Nest/FastAPI…)
  ├─ uploads + endpoints REST
  ├─ cola de jobs (BullMQ / Celery / Cloud Tasks)
  │    ├─ extract-pdf
  │    ├─ detect-barriers
  │    ├─ generate-proposal (LLM)
  │    └─ export-document
  └─ DB (Postgres) + storage (S3/MinIO) para PDFs
```

Una sola API HTTP al principio. Si más adelante el LLM o el PDF son pesados, podés separar workers; el contrato del front no cambia.

Resumen de cantidad

- **1 API** (REST)
- **~12 endpoints** cubren todo el flujo del frontend
- **1 canal de progreso** (polling o SSE) para Analizar/export
- **Persistencia en DB** por `analysisId` + `barrierId`

Lo que hoy es mock/`setTimeout`/`sessionStorage` se convierte en: crear análisis → consultar estado → CRUD de propuestas/decisiones → exportar.